Task 2 — Optimal DBSCAN eps via k-distance elbow

Goal: Find the optimal `eps` parameter for DBSCAN using the k-nearest-neighbor elbow method, run DBSCAN on both datasets, and verify the clusters visually.

Method follows the resources provided in the course material (`share.py`) and the original DBSCAN paper: Ester et al. (1996), KDD-96.

Step 1 - Remove ground

In [4]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN

# Load datasets
pcd1 = np.load("dataset1.npy")
pcd2 = np.load("dataset2.npy")

# Ground filter (histogram method from Task 1)
def get_ground_level(pcd):
    counts, bin_edges = np.histogram(pcd[:, 2], bins=200)
    peak_idx = np.argmax(counts)
    return bin_edges[peak_idx + 1] + 0.5

pcd1_above = pcd1[pcd1[:, 2] > get_ground_level(pcd1)]
pcd2_above = pcd2[pcd2[:, 2] > get_ground_level(pcd2)]

print(f"dataset1: {len(pcd1_above)} points above ground")
print(f"dataset2: {len(pcd2_above)} points above ground")
```

```
dataset1: 48676 points above ground
dataset2: 65608 points above ground
```

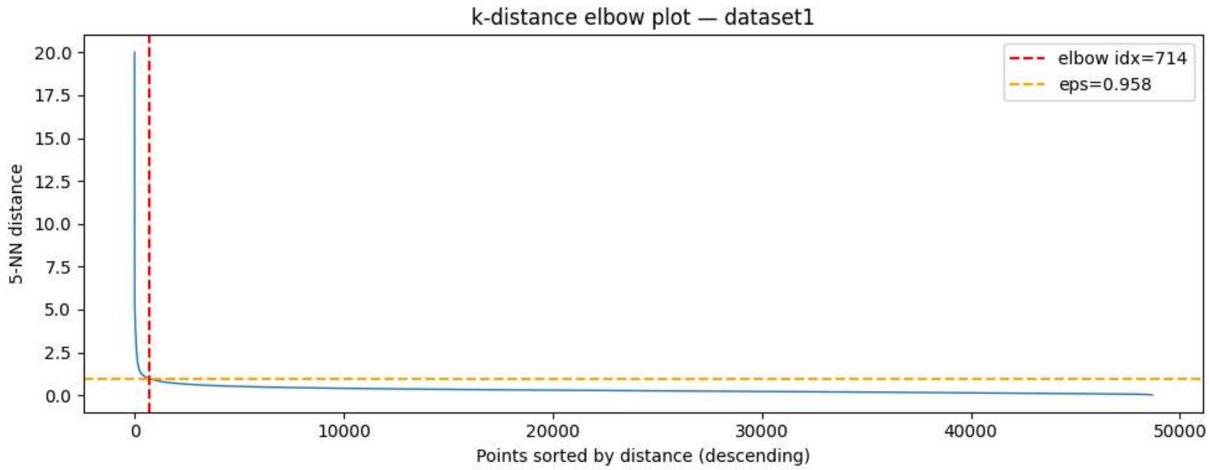
Step 2 - Find optimal eps using the k-distance elbow method

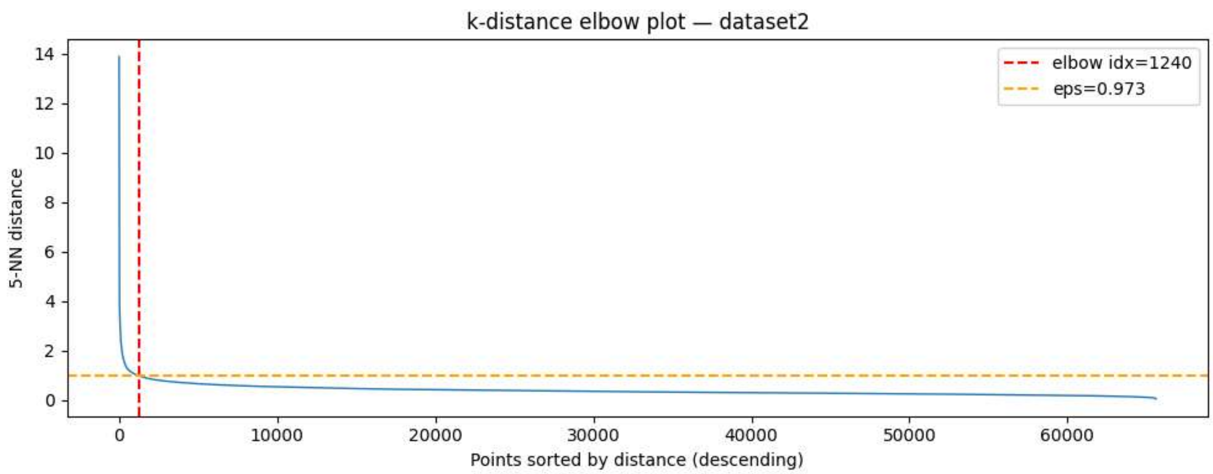
DBSCAN needs an eps value that defines how close two points must be to be considered neighbors. A value that is too large merges everything into one cluster; too small and most points become noise.

To find a good value, we compute the distance from each point to its 5th nearest neighbor (matching min_samples=5). Points inside a dense cluster have small distances; points at the edge of a cluster have larger ones.

Sorting these distances and plotting them reveals an "elbow" - a sharp bend where the curve transitions from flat (inside clusters) to steep (between clusters). That bend is the optimal eps.

In [5]:
```
def find_optimal_eps(pcd, k=5, title=""):
    # Fit k-NN and get distance to k-th nearest neighbor for every point
    nbrs = NearestNeighbors(n_neighbors=k + 1).fit(pcd)
    distances, _ = nbrs.kneighbors(pcd)
    k_dist = np.sort(distances[:, k])[::-1]  # descending order

    # Automatic elbow detection: point furthest from the line start→end
    n = len(k_dist)
    x = np.arange(n)
    # Line from first to last point
    p1 = np.array([0, k_dist[0]])
    p2 = np.array([n - 1, k_dist[-1]])
    line_vec = p2 - p1
    # Distance from each point on curve to that line
    point_vecs = np.column_stack([x, k_dist]) - p1
    line_len = np.linalg.norm(line_vec)
    cross = np.abs(np.cross(line_vec, point_vecs)) / line_len
    elbow_idx = np.argmax(cross)
    optimal_eps = k_dist[elbow_idx]

    # Plot
    plt.figure(figsize=(10, 4))
    plt.plot(k_dist, linewidth=1)
    plt.axvline(x=elbow_idx, color='red', linestyle='--', label=f"elbow idx={elbow_
    plt.axhline(y=optimal_eps, color='orange', linestyle='--', label=f"eps={optimal
    plt.xlabel("Points sorted by distance (descending)")
    plt.ylabel(f"{k}-NN distance")
    plt.title(f"k-distance elbow plot — {title}")
    plt.legend()
    plt.tight_layout()
    plt.show()

    return optimal_eps

eps1 = find_optimal_eps(pcd1_above, k=5, title="dataset1")
eps2 = find_optimal_eps(pcd2_above, k=5, title="dataset2")

print(f"Optimal eps — dataset1: {eps1:.3f}")
print(f"Optimal eps — dataset2: {eps2:.3f}")
```






```
Optimal eps — dataset1: 0.958
Optimal eps — dataset2: 0.973
```

Both datasets produce very similar optimal eps values: 0.958 for dataset1 and 0.973 for dataset2. This makes sense — they cover the same railway corridor, so the point density should be comparable.

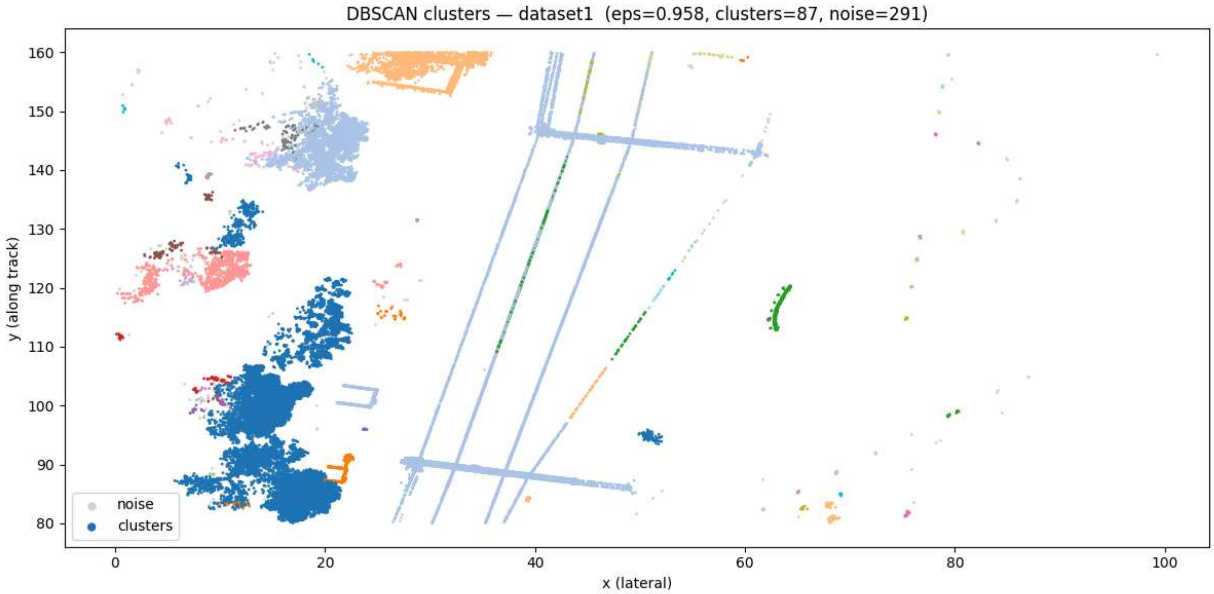
The elbow is detected early in both plots (index 714 and 1240 respectively), right where the curve transitions from steep to flat. The long flat tail represents the majority of points sitting inside dense clusters, all close to their neighbors. The steep initial drop comes from a small number of isolated or edge points with large neighbor distances.

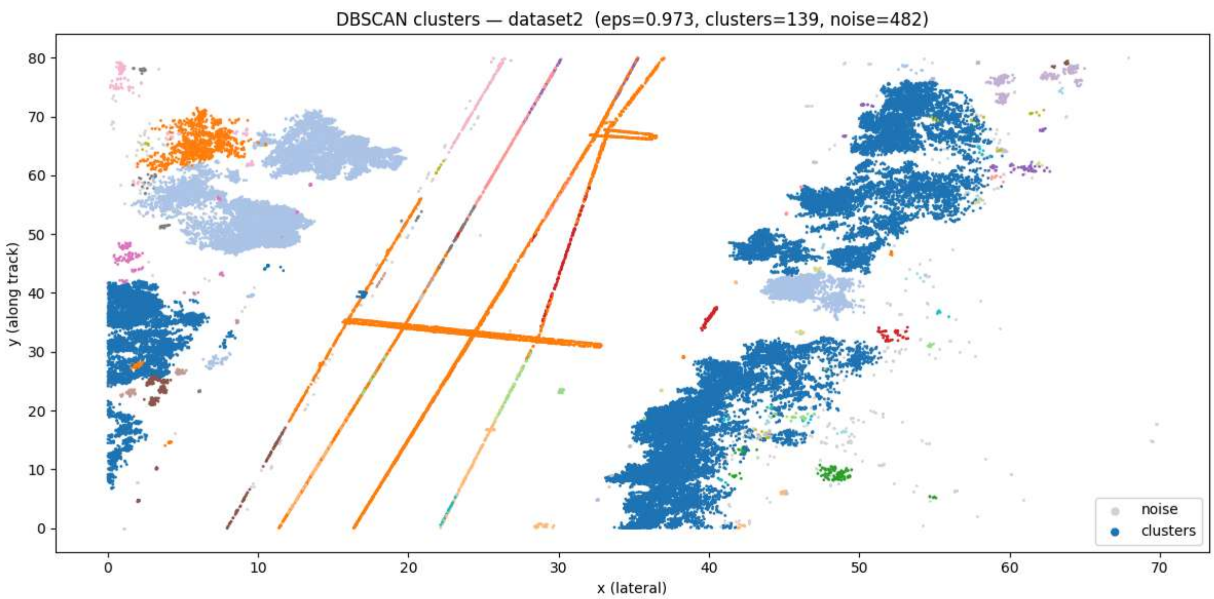
An eps around 0.96–0.97 means DBSCAN will consider two points as neighbors if they are within roughly one meter of each other. Although an eps value close to one meter initially seemed large compared to the physical size of railway objects such as cables, the value reflects the spacing between LiDAR points rather than the object dimensions themselves.

Step 3 - Run DBSCAN and inspect clusters

With the optimal eps values determined from the elbow plots, DBSCAN is applied to both datasets. The result is visualised as a top-down view (x vs y), showing the point cloud as seen from above. Noise points (label = −1) are shown in grey.

In [6]:
```
def run_dbscan(pcd, eps, title=""):
    clustering = DBSCAN(eps=eps, min_samples=5).fit(pcd)
    labels = clustering.labels_

    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = np.sum(labels == -1)

    # Plot — top-down view (x vs y)
    plt.figure(figsize=(12, 6))
    # Noise points in grey
    noise = pcd[labels == -1]
    plt.scatter(noise[:, 0], noise[:, 1], s=1, c='lightgrey', label='noise')
    # Cluster points colored by label
    mask = labels != -1
    plt.scatter(pcd[mask, 0], pcd[mask, 1], s=1,
                c=labels[mask], cmap='tab20', label='clusters')
    plt.title(f"DBSCAN clusters — {title}  (eps={eps:.3f}, clusters={n_clusters}, n
    plt.xlabel("x (lateral)")
    plt.ylabel("y (along track)")
    plt.legend(markerscale=5)
    plt.tight_layout()
    plt.show()

    return labels, n_clusters

labels1, n1 = run_dbscan(pcd1_above, eps1, title="dataset1")
labels2, n2 = run_dbscan(pcd2_above, eps2, title="dataset2")

print(f"dataset1: {n1} clusters")
print(f"dataset2: {n2} clusters")
```






```
dataset1: 87 clusters
dataset2: 139 clusters
```

Observations

Both datasets cluster good along the lines, and there is not much noise. The long diagonal linear clusters are consistent with the catenary wire running along the track. The larger irregular clusters to the sides is probably represent trees, bushes or som railway-connected installments. Dataset2 produces more clusters (139 vs 87), which is because odf more trees.